```
    // Invoke REST endpoint '/api/contacts' using http.post
  }
  deleteContact(id) {
    // Invoke REST endpoint '/api/contacts/<id>' using http.
delete
  }
}
```

Implementing the UI components

Let's now use Angular to implement the UI components. Also, let's use the *@angular-redux/store* module to interface Angular with Redux.

When generating the new project, Angular CLI would have created a component called *'App'*. Modify the HTML template *app.component.html* to render the *Contacts* component (*<app-contacts>*), which we will be implementing shortly.

To create the *Contacts* and the *Contact Form* component, run the Angular CLI *generate component* command from the folder *'app'*.

```
ng generate component contacts
ng generate component contactform
```

The above commands will create the folders, *contacts* and *contactform* under *app*, and generate the standard component blueprint containing the HTML template file, CSS file, component typescript file, and a component test specification file.

The *ContactsComponent* needs to access the Redux store to dispatch actions and to receive the state changes. For this, we inject NgRedux into the constructor of *ContactsComponent*, and using the NgRedux object, we can dispatch actions.

```
constructor(private ngRedux: NgRedux<any>, private _
contactActions: ContactActions) { }
```

In the *ngOnInit* life cycle method of *ContactsComponent*, we can then dispatch the action to load the contacts, which is implemented in *contactActions.ts* as explained in the 'Implementing the actions' section above.

```
  ngOnInit() {
    this.ngRedux.dispatch<any>(this._contactActions.
loadContacts());
  }
```

Additionally, in the *deleteContact* method of *ContactsComponent*, dispatch the action to delete the selected contact.

```
  deleteContact(id: any) {
    this.ngRedux.dispatch<any>(this._contactActions.
deleteContact(id));
  }
```

To receive the state changes from the store, we declare the *@select* decorator, and specify the portion of the state that the *ContactsComponent* is interested in, which is 'contacts'.

```
@select() contacts:any;
```

Whenever there is a state change, the Redux store will provide the latest state to the *ContactsComponent*, and due to data binding, the changes will reflect in the view.

Next, we will implement the *ContactformComponent*. This component also needs access to the store to dispatch an action, and hence we need to inject NgRedux in the constructor of *ContactformComponent*.

```
constructor(private ngRedux: NgRedux<any>, private _
contactActions: ContactActions) { }
```

The *ContactformComponent* will have the necessary HTML template to accept the user input to create the contact. When the user submits the contact information to be added, we will dispatch the action to add the contact.

```
  onSubmit(formValue: any) {
      this.ngRedux.dispatch<any>(this._contactActions.
addContact(newContact));
  }
```

Updating *styles.css* to include Bootstrap

In the *styles.css* file, under the *src* folder, add the link to the Bootstrap CSS for a presentable UI:

```
@import "https://cdnjs.cloudflare.com/ajax/libs/twitter-
bootstrap/3.3.7/css/bootstrap.min.css"
```

Running the application

1. Go to the Node.js command prompt.
2. Change directory to *contacts/server* and start the server:

```
npm start
```

3. Change the directory to *contacts/client*. Start the Angular *Contacts* app:

```
npm start
```

This will start the application and display the *Contacts* information. We can now add and delete contacts. END

By: Srinivasan M.

The author works at Wipro and has experience in various JavaScript frameworks. He can be reached at *srinivm.srinivasan@yahoo.co.in*.